IT STUDY HUB

COMPLETE C++ PROGRAMMING SERIES

MODULE 4: ARRAYS, STRINGS & POINTER BASICS

Authors: Mithila Rani Saha & Dithsa Dutta

Website: www.itstudyhub.dpdns.org

Module Overview

1. About This Module

Welcome to **Module 4: Arrays, Strings & Pointer Basics** in the IT Study Hub Complete C++ Programming Series. Building upon the foundational control structures and modular functions mastered in Module 3, this module unlocks data aggregation and memory manipulation. Authored by Mithila Rani Saha and Dithsa Dutta, this handbook guides you through continuous

memory blocks (arrays), text handling (C-style strings and `std::string`), low-level memory addresses via pointers and references, and dynamic memory allocation (`new` and `delete`).

2. Learning Outcomes

By completing this module, you will be able to:

- Declare, initialize, traverse, and manipulate 1D and multidimensional arrays.
- Process text using both legacy C-style character arrays and modern `std::string` objects.
- Master pointer fundamentals including the address-of operator (`&`), dereference operator (`*`), and pointer arithmetic.
- Differentiate between pointers, reference variables, and `const` pointer variations.
- Allocate and deallocate memory dynamically on the heap using `new` and `delete`, preventing memory leaks and dangling pointers.

3. Module Coverage

This module comprehensively covers:

- **Array Data Structures:** One-dimensional arrays, initialization, traversal, and multidimensional matrices.
- **String Manipulation:** Character arrays, null termination, `std::string`, comparison, concatenation, and I/O.
- **Memory Addressing:** Pointers, reference variables, pointer-to-array relationships, and function parameter passing.
- **Dynamic Memory:** Stack vs Heap architecture, `new`/`delete` lifecycles, and safe memory practices.
- **Hands-On Project:** Building an advanced Student Marks Analysis System.
- **Rigorous Assessments:** 15+ practice questions, 20+ viva questions with answers, 15+ technical interviews, and coding challenges.

4. Main Notes: Theoretical Foundations

Memory management and data aggregation are critical pillars of systems programming. Below are the exhaustive foundational notes for Module 4.

Concept: One-Dimensional Arrays, Initialization, and Traversal

Simple Explanation: A one-dimensional array is a contiguous block of fixed-size memory locations that store multiple elements of the identical data type under a single variable name, accessible via integer indices starting from 0.

> ***Real-Life Analogy:*** *Think of a 1D array as a row of numbered post office boxes or hotel lockers. Each box holds one item, and you access a specific box directly using its locker number (index).*

> **Syntax:**
> **`datatype arrayName[size];`**
> **`datatype arrayName[size] = {val1, val2, val3};`**

```
#include
using namespace std;

int main() {
    int scores[5] = {85, 90, 78, 92, 88};

    cout << "Traversing array elements: " << endl;
    for(int i = 0; i < 5; i++) {
        cout << "Element at index " << i << ": " << scores[i] <<
endl;
    }
```

```
    return 0;
}
```

```
Output:
Traversing array elements:
Element at index 0: 85
Element at index 1: 90
Element at index 2: 78
Element at index 3: 92
Element at index 4: 88
```

Line-by-Line Breakdown:

- **int scores[5] = {...};**: Allocates contiguous RAM space for 5 integers.
- **for(int i = 0; i < 5; i++)**: Iterates through valid indices from 0 to 4.
- **scores[i]**: Accesses the element at the current index.

Real-World Application: Storing daily temperature readings or stock prices over a 30-day month in a continuous performance buffer.

Common Mistakes: Accessing an array out of bounds (e.g., `scores[5]` when valid indices are 0 to 4), causing undefined behavior or memory corruption.

Best Practices: Always ensure loop bounds match the exact array size. In modern C++, prefer `std::vector` when array sizes must be dynamic.

Actionable Practice: Write an array of 5 integers, take user input to populate them, and find their mathematical sum.

Concept: Multidimensional Arrays

Simple Explanation: A multidimensional array is an array of arrays (e.g., 2D matrices structured in rows and columns). Data is stored contiguously in memory row-by-row.

> ***Real-Life Analogy:*** *Think of a 2D array as a spreadsheet grid, a chessboard, or theater seating where rows and column coordinates pinpoint a precise seat.*

Syntax:

```
datatype arrayName[rows][cols];
datatype matrix[2][3] = {{1, 2, 3}, {4, 5, 6}};
```

```
#include
using namespace std;

int main() {
    int matrix[2][3] = {
        {1, 2, 3},
        {4, 5, 6}
    };

    for(int r = 0; r < 2; r++) {
        for(int c = 0; c < 3; c++) {
            cout << matrix[r][c] << " ";
        }
        cout << endl;
    }
    return 0;
}
```

Output:

```
1 2 3
4 5 6
```

Line-by-Line Breakdown:

- **int matrix[2][3]**: Declares a 2D grid with 2 rows and 3 columns.
- Nested loops traverse every row `r` and column `c` sequentially.

Real-World Application: Representing game boards (Tic-Tac-Toe, Chess), pixel grids in digital image processing, and mathematical matrices.

> **Common Mistakes:** Forgetting to specify the column dimension size when passing a 2D array into a function signature.

> **Best Practices:** Always maintain clear row and column boundary constants to prevent index out-of-bound errors during matrix traversals.

> **Actionable Practice:** Write a program that adds two 2x2 matrices and prints the resulting matrix.

Concept: Character Arrays, C-Style Strings, and std::string

Simple Explanation: C-style strings are null-terminated (`\0`) character arrays inherited from C. Modern C++ introduces `std::string` from the `` library, providing dynamic memory management, safety, and powerful built-in methods.

> ***Real-Life Analogy:*** *A C-style string is like an old train carriage where cars are linked together and terminated with a special red caboose flag (`\0`). `std::string` is like a modern smart-length vehicle that automatically adjusts its seating capacity as passengers enter and leave.*

Syntax:

```
char cStr[] = "Hello";
string modernStr = "Hello C++";
```

```
#include
#include
using namespace std;

int main() {
    char greeting[] = "Hello"; // C-style string
    string name = "IT Study Hub"; // std::string

    cout << greeting << " from " << name << endl;
    cout << "String length: " << name.length() << endl;
    return 0;
}
```

Output:

```
Hello from IT Study Hub
String length: 12
```

Line-by-Line Breakdown:

- **char greeting[]**: Automatically sized to 6 bytes to accommodate 'H','e','l','l','o','\0'.
- **string name**: Uses the STL string class for high-level text manipulation.
- **name.length()**: Returns the character count.

Real-World Application: Processing user text input, filenames, and network protocol headers.

Common Mistakes: Forgetting the null-terminating character `\0` when manually initializing char arrays, leading to buffer overruns during printing.

Best Practices: Always prefer `std::string` over raw C-style character arrays in modern C++ applications for safety and flexibility.

> **Actionable Practice:** Take two separate strings for first and last name, concatenate them, and print the full name.

Concept: String Operations, Methods, Comparison, and Concatenation

Simple Explanation: The `std::string` class provides rich built-in operations including concatenation (`+`), comparison (`==`, `compare()`), searching (`find()`), substring extraction (`substr()`), and appending.

> ***Real-Life Analogy:*** *String operations are like digital text editing tools in a word processor: searching for keywords, stitching paragraphs together (concatenation), and measuring word counts.*

> **Syntax:**
> **`string s3 = s1 + s2;`**
> **`bool match = (s1 == s2);`**
> **`size_t pos = s1.find("sub");`**

```
#include
#include
using namespace std;

int main() {
    string s1 = "IT Study";
    string s2 = " Hub";
    string full = s1 + s2; // Concatenation

    cout << "Full: " << full << endl;
    cout << "Find 'Study': index " << full.find("Study") << endl;
    cout << "Substring: " << full.substr(3, 5) << endl;
```

```
    return 0;
}
```

```
Output:
Full: IT Study Hub
Find 'Study': index 3
Substring: Study
```

Line-by-Line Breakdown:

- **s1 + s2**: Concatenates two strings together.
- **full.find("Study")**: Returns the starting index where 'Study' appears.
- **full.substr(3, 5)**: Extracts 5 characters starting at index 3.

Real-World Application: Parsing configuration files, URL routing, and text search algorithms in web engines.

Common Mistakes: Using `cin >> stringVar` to read multi-word sentences containing spaces (it stops at the first whitespace; use `getline(cin, var)` instead).

Best Practices: Use `getline()` when accepting full-line text input from users.

Actionable Practice: Write a program that takes a sentence and counts how many characters it contains.

Concept: Pointers: Address Operator (&) and Dereference Operator (*)

Simple Explanation: A pointer is a variable that stores the memory address of another variable rather than its direct value. The address-of operator (`&`) retrieves a variable's memory address, while the dereference operator (`*`) accesses the value stored at that address.

> ***Real-Life Analogy:*** *Think of a regular variable as a house containing furniture (the value). A pointer is like the GPS coordinate or home address written on a slip of paper. The `&` operator looks up your house's GPS address, and the `*` operator drives to that address to inspect what furniture is inside.*

Syntax:

```
datatype *pointerName = &variable;
*pointerName = newValue; // Dereferencing assignment
```

```
#include
using namespace std;

int main() {
    int age = 25;
    int *ptr = &age; // Pointer stores address of age

    cout << "Value of age: " << age << endl;
    cout << "Address of age (&age): " << &age << endl;
    cout << "Value stored in ptr: " << ptr << endl;
    cout << "Dereferenced value (*ptr): " << *ptr << endl;
    return 0;
}
```

Output:

```
Value of age: 25
Address of age (&age): 0x7ffee4a (example address)
```

```
Value stored in ptr: 0x7ffee4a
Dereferenced value (*ptr): 25
```

Line-by-Line Breakdown:

- **int *ptr = &age;**: Declares an integer pointer `ptr` and assigns it the memory address of `age`.
- ***ptr**: Dereferences the pointer to access or modify the underlying value at that address.

Real-World Application: Low-level hardware interfacing, operating system kernel development, and efficient data structure traversal (linked lists, trees).

Common Mistakes: Using an uninitialized pointer (wild pointer) or accessing a pointer pointing to deleted memory (dangling pointer), causing segmentation faults.

Best Practices: Always initialize pointers immediately upon declaration (assign `nullptr` if no address is available yet).

Actionable Practice: Declare a double variable, create a pointer to it, and modify its value using the dereference operator.

Concept: Pointer Arithmetic and Pointers with Arrays

Simple Explanation: Pointer arithmetic allows incrementing (`++`) or decrementing (`--`) pointer addresses based on the byte size of the underlying data type. Array names inherently decay into constant pointers pointing to their first element (`&arr[0]`), establishing a deep relationship between arrays and pointers.

> ***Real-Life Analogy:*** *Imagine walking down a row of lockers where each locker is numbered sequentially. Pointer arithmetic is like stepping from locker 1 to locker 2 by adding the exact physical width of one locker to your step count.*

```
Syntax:
int arr[3] = {10, 20, 30};
int *p = arr; // points to arr[0]
*(p + 1); // accesses arr[1]
```

```
#include
using namespace std;

int main() {
    int nums[] = {10, 20, 30, 40};
    int *p = nums;

    cout << "Using pointer arithmetic:" << endl;
    for(int i = 0; i < 4; i++) {
        cout << "Element " << i << " = " << *(p + i) << endl;
    }
    return 0;
}
```

```
Output:
Using pointer arithmetic:
Element 0 = 10
Element 1 = 20
Element 2 = 30
Element 3 = 40
```

Line-by-Line Breakdown:

- **int *p = nums;**: Array name `nums` decays into a pointer to `nums[0]`.
- ***(p + i)**: Adds `i` multiplied by `sizeof(int)` to the address and dereferences it.

Real-World Application: High-performance image filters and audio stream buffers use pointer arithmetic to zip through raw memory arrays at maximum CPU speed.

Common Mistakes: Stepping a pointer outside allocated array boundaries via arithmetic, leading to unauthorized memory reads.

Best Practices: Keep pointer arithmetic bounds tightly checked against known array lengths.

Actionable Practice: Traverse an array of characters backward using pointer subtraction and decrementing.

Concept: Pointers and Functions, References, and Const Pointers

Simple Explanation: Functions can accept pointers or reference variables (`&`) as parameters to achieve pass-by-reference. Reference variables act as constant aliases to existing variables. `const` modifiers can lock pointer targets or pointer values.

> ***Real-Life Analogy:*** *A reference is like having a nickname for a person—both names refer to the exact same living human. A `const` pointer is like a locked compass that points to a specific destination but cannot be turned away from it.*

Syntax:

```
void func(int *p); // Pass by pointer
void func(int &r); // Pass by reference
const int *p; // Pointer to const data
int * const p = &x; // Const pointer to non-const data
```

```
#include
using namespace std;

void tripleValue(int *val) {
```

```
    *val *= 3;
}

int main() {
    int x = 10;
    tripleValue(&x);
    cout << "Tripled x: " << x << endl;
    return 0;
}
```

Output:
```
Tripled x: 30
```

Line-by-Line Breakdown:

- **tripleValue(&x)**: Passes the memory address of `x` into the function.
- ***val *= 3**: Dereferences the pointer and triples the original variable directly.

Real-World Application: Passing large data structures into graphics rendering functions without copying heavy data payload blocks.

Common Mistakes: Returning references or pointers to local variables that get destroyed when the function exits (dangling reference/pointer).

Best Practices: Prefer C++ references (`&`) over pointers when nullability is not required, as references are safer and cleaner.

Actionable Practice: Write a function that takes two integer pointers and swaps their values.

Concept: Dynamic Memory Basics: new and delete, Stack vs Heap

Simple Explanation: Static memory is automatically allocated on the stack. Dynamic memory is allocated at runtime on the heap using the `new` operator and must be explicitly released using `delete` to prevent memory leaks.

> ***Real-Life Analogy:*** *The stack is like a hotel room with automated housekeeping—things are cleaned up automatically when you check out (out of scope). The heap is like renting an empty warehouse space: you must personally call the cleanup crew (`delete`) when you are done, otherwise you pay for abandoned clutter forever (memory leak).*

```
Syntax:
int *ptr = new int(50);
// use ptr
delete ptr;
ptr = nullptr;

int *arr = new int[10];
delete[] arr;
```

```
#include
using namespace std;

int main() {
    int *pNum = new int(100); // Allocate on heap

    cout << "Dynamic heap value: " << *pNum << endl;

    delete pNum; // Free memory
    pNum = nullptr; // Prevent dangling pointer

    cout << "Memory successfully released." << endl;
    return 0;
}
```

```
Output:
Dynamic heap value: 100
Memory successfully released.
```

Line-by-Line Breakdown:

- **new int(100)**: Allocates dynamic integer memory on the heap and initializes it to 100.
- **delete pNum;**: Returns heap memory back to the operating system.
- **pNum = nullptr;**: Resets pointer to null, preventing accidental dangling pointer access.

Real-World Application: Video games allocating dynamic enemy objects and particle systems on the fly as players progress through levels.

Common Mistakes: Forgetting to call `delete` (Memory Leak), or calling `delete` twice on the same pointer (Double Free corruption).

Best Practices: In Modern C++ (C++11 and beyond), prefer Smart Pointers (`std::unique_ptr`, `std::shared_ptr`) over raw `new` and `delete` to automate heap memory management safely.

Actionable Practice: Allocate a dynamic array of 5 integers on the heap using `new[]`, populate it, print elements, and release it using `delete[]`.

Visual Workflows & Architecture

Architectural diagrams mapping out continuous array layouts, memory address dereferencing, pointer arithmetic, and heap allocation lifecycles.

Diagram 1: Array Memory Layout (Contiguous RAM)



Diagram 2: Pointer → Address → Value Relationship

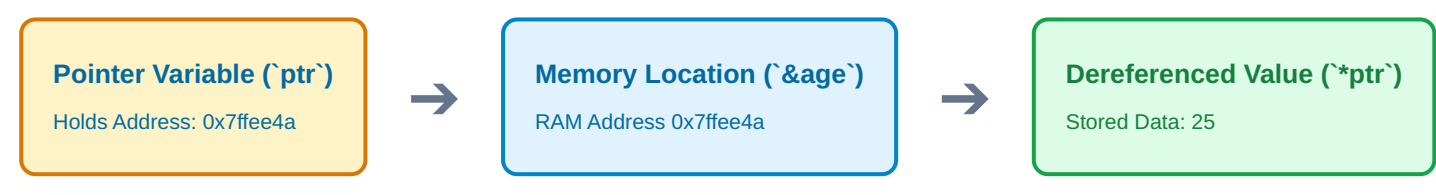
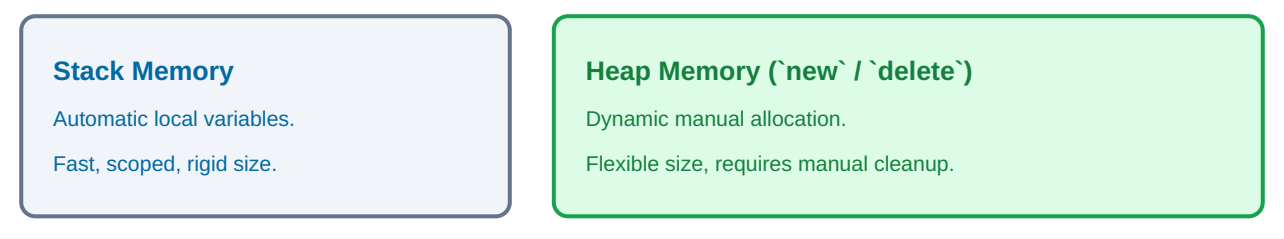


Diagram 3: Stack vs Heap Memory Architecture



Comparative Analysis

Professional C++ development requires choosing the correct data structure and memory management technique for optimal performance and safety.

Table 1: Array vs std::vector

Feature	Built-in Array (`int arr[10]`)	`std::vector` (`std::vector`)
Size	Fixed size (determined at compile time)	Dynamic size (resizes automatically)
Safety	No bounds checking (risk of overflow)	Bounds checking available via `.at()`
Memory	Allocated on Stack (usually)	Allocated on Heap (manages internal buffer)

Table 2: C-Style String vs std::string

Aspect	C-Style String (`char[]`)	`std::string`
Termination	Requires explicit null-terminator (`\0`)	Managed automatically
Operations	Requires C-string library functions (`strcpy`, `strlen`)	Rich operator support (`+`, `==`, `.find()`, `.substr()`)
Safety	Prone to buffer overflow vulnerabilities	Highly secure and robust

Table 3: Pointer vs Reference

Feature	Pointer (`int*`)	Reference (`int&`)
Nullability	Can be assigned `nullptr`	Cannot be null; must bind to an object
Reassignment	Can be pointed to different variables	Cannot be reseated once bound
Syntax	Requires explicit `&` and `*` operators	Used naturally like a regular variable

Table 4: Stack vs Heap

Aspect	Stack Memory	Heap Memory
Lifetime	Automatic (destroyed when leaving scope)	Manual (persists until `delete` or program exit)
Speed	Extremely fast allocation	Slower allocation via OS memory manager
Risk	Stack overflow if recursion is too deep	Memory leaks, dangling pointers, fragmentation

Table 5: Const Pointer Variations

Declaration Syntax	What is Const?	Can Pointer be Reseated?	Can Value be Modified?
`const int *p`	Pointer to Constant Data	Yes	No
`int * const p`	Constant Pointer to Data	No	Yes
`const int * const p`	Constant Pointer to Constant Data	No	No

HANDS-ON PROJECT: STUDENT MARKS ANALYSIS SYSTEM

In this comprehensive hands-on project, we combine arrays, strings, functions, references, pointers, and dynamic memory allocation into a professional student performance analytics utility.

Project Specification

Problem Statement: Build an interactive Student Marks Analysis System that stores student names and test scores across multiple subjects using arrays and strings. The system must compute total scores, class averages, find highest and lowest marks, search for individual student records, and demonstrate dynamic memory allocation for expandable test records.

```
#include <iostream>
using namespace std;

// Function Prototypes
void calculateStats(const int marks[], int size, int &total, double
&average, int &highest, int &lowest);
void searchStudent(const string names[], const int marks[], int
size, string targetName);
void displayRecords(const string names[], const int marks[], int
size);

int main() {
    const int numStudents = 5;
    string studentNames[numStudents] = {"Mithila", "Dithsa",
"Aarav", "Priya", "Rahul"};
    int studentMarks[numStudents] = {88, 92, 79, 95, 84};

    cout << "=======================================" << endl;
    cout << "   IT STUDY HUB STUDENT MARKS SYSTEM    " << endl;
    cout << "=======================================" << endl;

    displayRecords(studentNames, studentMarks, numStudents);
```

```
    int total = 0, highest = 0, lowest = 0;
    double average = 0.0;

    calculateStats(studentMarks, numStudents, total, average,
highest, lowest);

    cout << "\n--- Class Performance Analytics ---" << endl;
    cout << "Total Marks Sum : " << total << endl;
    cout << "Class Average   : " << average << endl;
    cout << "Highest Score   : " << highest << endl;
    cout << "Lowest Score    : " << lowest << endl;

    // Searching student
    string searchKey = "Priya";
    cout << "\nSearching record for: " << searchKey << endl;
    searchStudent(studentNames, studentMarks, numStudents,
searchKey);

    // Dynamic Memory Demonstration
    cout << "\n--- Dynamic Memory Heap Allocation ---" << endl;
    int dynSize;
    cout << "Enter number of extra dynamic scores to allocate: ";
    cin >> dynSize;

    int *dynamicScores = new int[dynSize];
    for(int i = 0; i < dynSize; i++) {
        dynamicScores[i] = (i + 1) * 10;
        cout << "Heap Score " << i + 1 << " = " << dynamicScores[i]
<< endl;
    }

    // Free heap memory safely
    delete[] dynamicScores;
    dynamicScores = nullptr;
    cout << "Dynamic heap memory successfully released." << endl;

    return 0;
}

// Function Definitions
void displayRecords(const string names[], const int marks[], int
size) {
    cout << "Student Records:\n";
    for(int i = 0; i < size; i++) {
        cout << i + 1 << ". " << names[i] << " \t- Score: " <<
marks[i] << endl;
```

```
    }
}

void calculateStats(const int marks[], int size, int &total, double
&average, int &highest, int &lowest) {
    total = 0;
    highest = marks[0];
    lowest = marks[0];

    for(int i = 0; i < size; i++) {
        total += marks[i];
        if(marks[i] > highest) highest = marks[i];
        if(marks[i] < lowest) lowest = marks[i];
    }
    average = static_cast(total) / size;
}

void searchStudent(const string names[], const int marks[], int
size, string targetName) {
    bool found = false;
    for(int i = 0; i < size; i++) {
        if(names[i] == targetName) {
            cout << "Found! Student: " << names[i] << " | Score: "
<< marks[i] << endl;
            found = true;
            break;
        }
    }
    if(!found) {
        cout << "Student not found in records." << endl;
    }
}
```

Expected Output:

```
========================================
   IT STUDY HUB STUDENT MARKS SYSTEM
========================================
Student Records:
1. Mithila      - Score: 88
2. Dithsa       - Score: 92
3. Aarav        - Score: 79
4. Priya        - Score: 95
```

```
5. Rahul        - Score: 84

--- Class Performance Analytics ---
Total Marks Sum : 438
Class Average   : 87.6
Highest Score   : 95
Lowest Score    : 79

Searching record for: Priya
Found! Student: Priya | Score: 95

--- Dynamic Memory Heap Allocation ---
Enter number of extra dynamic scores to allocate: 3
Heap Score 1 = 10
Heap Score 2 = 20
Heap Score 3 = 30
Dynamic heap memory successfully released.
```

Step-by-Step Explanation:

- Parallel arrays (`studentNames` and `studentMarks`) store student identities and corresponding grades.
- `calculateStats` utilizes pass-by-reference parameters (`int &total`, `double &average`) to return multiple computed analytics back to `main()`.
- `searchStudent` iterates through arrays to locate specific matches safely.
- Dynamic heap allocation (`new int[dynSize]` and `delete[]`) demonstrates runtime memory management with proper pointer nullification.

> **Practice Challenges:**
>
> - Extend the system to sort students in descending order based on their marks.
> - Convert the parallel arrays into a struct-based collection (`struct Student { string name; int mark; }`).

Module Assessments

Test your mastery of arrays, strings, pointers, and dynamic memory with these rigorous assessment questions.

15+ Practice Questions

1. What is the significance of index 0 in C++ arrays?
2. Write a program that declares an integer array of size 10 and finds the maximum element.
3. Why do C-style strings require a null-terminator (`\0`)?
4. Differentiate between `char greeting[] = "Hello";` and `string s = "Hello";`.
5. What is the output of `sizeof(arr)` when passed into a function as a parameter? (Hint: Pointer decay).
6. Explain the relationship between array names and pointers.
7. What is the difference between the address-of operator (`&`) and the dereference operator (`*`)?
8. What is a dangling pointer, and how does it occur?
9. How do you allocate and deallocate a dynamic array on the heap?
10. Explain pass-by-reference using reference variables vs pointers.
11. Write a function that reverses an array of integers in place using pointers.
12. What is a memory leak, and what happens if you forget to call `delete[]`?
13. Differentiate between stack memory and heap memory allocation.
14. Explain the difference between `const int *p` and `int * const p`.
15. Why is `getline(cin, str)` preferred over `cin >> str` for reading sentences?

20+ Viva Questions (With Answers)

Q1: What is an array?

> A contiguous block of memory storing multiple elements of the same data type under a single identifier.

Q2: What is array indexing start value in C++?

> Index starts at 0.

Q3: What is array decay?

> The automatic conversion of an array name into a pointer pointing to its first element when passed to a function.

Q4: What is a C-style string?

> An array of characters terminated by a null character (`\0`).

Q5: What library is required for `std::string`?

> `#include <string>`.

Q6: What does the address-of operator (`&`) do?

> Returns the memory address RAM location of a variable.

Q7: What does the dereference operator (`*`) do?

> Accesses or modifies the value stored at a specific memory address pointed to by a pointer.

Q8: What is a pointer?

> A variable that stores the memory address of another variable.

Q9: What is pointer arithmetic?

> Incrementing or decrementing pointer addresses based on the data type's byte size.

Q10: What is a reference variable?

> An alias to an existing variable that shares the same memory address.

Q11: Can a reference be null?

> No, references must always bind to a valid object upon declaration.

Q12: What is stack memory?

> Automatic memory used for local variables, managed automatically by the scope lifecycle.

Q13: What is heap memory?

> Dynamic memory allocated at runtime using `new` and released using `delete`.

Q14: What is a memory leak?

> Unreleased heap memory that remains allocated even after it is no longer needed.

Q15: What is a dangling pointer?

> A pointer that points to memory that has already been deallocated or freed.

Q16: How do you free a dynamic array allocated with `new[]`?

> Using the `delete[]` operator.

Q17: What is `nullptr`?

> A type-safe null pointer literal representing an unassigned pointer address.

Q18: What is the difference between `const int *p` and `int * const p`?

> `const int *p` points to constant data (value cannot change); `int * const p` is a constant pointer (address cannot change).

Q19: What is a multidimensional array?

> An array of arrays, such as a 2D matrix with rows and columns.

Q20: Why are smart pointers preferred over raw pointers in modern C++?

> They automate heap memory management, preventing memory leaks and double-free errors.

15+ Technical Interview Questions

1. **Why does passing an array to a function cause array decay, and how do you determine its size inside the function?** Explain pointer decay and passing size explicitly as an argument.
2. **What is the difference between `sizeof(arr)` and `sizeof(ptr)` where `arr` is an array and `ptr` is a pointer to it?** Contrast total static buffer bytes vs pointer address width (4 or 8 bytes).

3. **Explain the dangers of buffer overflows with C-style strings and how `std::string` mitigates them.** Discuss automatic capacity expansion vs fixed-size stack arrays.
4. **What is a double-free bug, and what security vulnerabilities does it introduce?** Explain heap corruption and exploit vectors.
5. **How do smart pointers (`std::unique_ptr`, `std::shared_ptr`) work under the hood? (Preview)** Discuss RAII (Resource Acquisition Is Initialization) and reference counting.
6. **Can you perform pointer arithmetic on `void*` pointers? Why or why not?** Explain that `void*` lacks a defined data type size, preventing byte-stride calculation without casting.
7. **What is a memory leak, and what profiling tools (like Valgrind or AddressSanitizer) detect it?** Discuss heap auditing techniques.
8. **Explain the difference between shallow copy and deep copy when pointers are involved in custom classes.** Discuss dangling references caused by default shallow copies.
9. **Why are references preferred over pointers in function parameter passing when nullability is not required?** Contrast syntax cleanliness, safety, and lack of null dereference risk.
10. **What is the purpose of `nullptr` over the legacy `NULL` macro in C++?** Explain type safety and elimination of integer ambiguity (`0` vs `NULL` vs `nullptr`).
11. **How do 2D arrays reside in physical memory? Is row-major or column-major order used in C++?** Explain row-major contiguous layout.
12. **What happens when you dereference a null pointer or wild pointer?** Explain operating system traps and Segmentation Fault (`SIGSEGV`) crashes.
13. **What is placement `new`, and when is it used in high-performance systems? (Preview)** Discuss constructing objects at specific pre-allocated memory addresses.
14. **How does pointer aliasing impact compiler optimization?** Explain how the optimizer assumes different pointers might point to the same memory location, restricting reordering.
15. **What is the Rule of Three / Rule of Five in C++ memory management? (Preview)** Discuss destructor, copy constructor, and copy assignment operators.

Module 4 Wrap-Up

Key Terms Glossary

- **Array:** A contiguous block of memory storing elements of identical type.
- **Null Terminator (`\0`):** The character marking the end of a C-style string.
- **Pointer:** A variable holding a memory address.
- **Dereference (`*`):** Accessing the value stored at a pointer's address.
- **Reference (`&`):** An immutable memory alias for an existing variable.
- **Heap:** Dynamic memory region managed via `new` and `delete`.
- **Memory Leak:** Unreleased heap memory causing performance degradation.

Quick Revision Summary

Module 4 bridges data structures and low-level memory architecture. One-dimensional and multidimensional arrays provide continuous storage. Strings range from C-style null-terminated character arrays to powerful `std::string` objects. Pointers (`&`, `*`, pointer arithmetic) interface directly with RAM addresses. References provide safe variable aliases, while heap allocation (`new`/`delete`) enables dynamic runtime memory management.

Pointer and Array Coding Challenges

Complete these challenges before moving to Module 5:

- **Challenge 1: Pointer Array Reverse.** Write a function that accepts an integer array pointer and its size, reversing the array in place using pointer arithmetic.
- **Challenge 2: Dynamic Matrix Allocator.** Write a program that asks the user for row and column dimensions, allocates a 2D matrix dynamically on the heap using pointers-to-pointers (`int**`), populates it, prints it, and frees all memory cleanly.
- **Challenge 3: Substring Finder.** Write a function using pointer traversal to check if a C-style string exists as a substring inside another string without using `` library functions.

CONGRATULATIONS ON COMPLETING MODULE 4!

WHAT COMES NEXT:

MODULE 5 — OBJECT-ORIENTED PROGRAMMING